

Multi-Tenant Chatbot Service Production Architecture and Operating

Model Date: 2025-08-10 UTC

Multi-Tenant Chatbot Service Production Architecture and Operating

Model Date: 2025-08-10 UTC

One-Page Summary Goal Deliver a secure, multi-tenant conversational AI platform that serves low-latency, high-reliability chat across web, mobile, and API clients. The system must support model orchestration, retrieval augmented generation, tool use, safety filtering, and analytics, with strong privacy and governance.

Key Outcomes

P95 time to first token under 300 ms for cached responses and under 800 ms for RAG assisted prompts in-region. High availability with 99.95 percent read and 99.9 percent write SLAs per region; graceful degradation paths. Robust tenant isolation, per-tenant customization of models, prompts, tools, and data sources. Operational excellence with canary releases, auto-rollback, and deep observability from prompt to token stream.

Primary Assumptions

Traffic ranges from thousands to hundreds of thousands of requests per second globally with bursty diurnal patterns. Average output lengths vary widely; token rate budgeting and admission control required. Multiple model backends are supported including first-party GPU clusters and third-party providers. Data sources for retrieval include object stores, vector indexes, and SaaS connectors.

Functional Requirements

Core Streaming chat completions with function or tool calls, images or files optional; conversation history, system prompts, and session level parameters. Retrieval Per-tenant knowledge bases, chunking and embedding pipelines, vector search with recency and metadata filters. Orchestration Policy driven router chooses model, temperature, max tokens, and tool set per request; supports fallbacks and canaries. Safety Prompt and output filters, PII redaction, toxicity checks, jailbreak detection; configurable per tenant. Customization Per-tenant system prompts, tools, skill graphs, guardrails, and rate limits; fine-tune or adapters optional. Billing and Quotas Usage metering by tokens and tool calls; budgets, hard caps, and alerts. Analytics Latency, token usage, containment rate, safety events, routing decisions, answer quality signals. Admin Tenant lifecycle, API keys, access policies, secrets, data connectors, and RBAC.

Non-

Availability 99.95 percent for read paths and streaming; 99.9 percent for writes and admin. Latency budgets p95: router 15 ms, auth 5 ms, safety prefilter 20 ms, retrieval 100-250 ms, model queue wait 0-150 ms at target utilization, decode to first token 50-200 ms, stream token rate 30-150 tokens per second. Durability RPO 0 for requests accepted into queues; RTO under 15 minutes per region. Security and privacy SOC 2 Type 2, GDPR, CCPA; data minimization, residency, and encryption with customer managed keys optional. Scalability horizontal via stateless frontends, sharded retrieval, and pooled model replicas; cost aware autoscaling.

Architecture Overview

Cell Architecture Each region is an independent cell: Edge, API Gateway, Router, Safety, Retrieval, Model Serving, Tooling, Billing, Observability. A thin global plane manages identity, configuration, catalogs, and quotas. Data Flow Client → Edge → API Gateway (auth, quota) → Router (policy, AB) → Safety Prefilter → Retrieval Fanout → Merge Context → Model Serving (stream) → Safety Postfilter → Tool Invocations if any → Stream back to client → Usage metering and logs. Degradation If retrieval or tools degrade, answer with model-only prompt using cached knowledge; fallback models for

overload or outages.

Technology Stack (reference) Edge and Auth: CDN, WAF, API Gateway, OAuth 2.1 or OpenID Connect, mTLS in mesh, HMAC for webhook signatures. Orchestration: Router written in Go or Rust; feature flags and AB via config service; policy engine OPA for request gating. Retrieval: Embedding workers, chunkers;

vector DB such as Milvus, pgvector, or OpenSearch k■NN; metadata store Postgres; document store S3 or GCS with Iceberg or Delta. Model Serving: GPU pools with vLLM or Triton; tokenizer service; KV cache for attention reuse; speculative decoding where supported; third■party model providers via adapters. Safety: Rule and ML filters; prompt hardening; redactors; output throttles; blocklists; per■tenant policy sets. Tooling: gRPC tools registry; serverless functions for lightweight tools; long■running tools via queues and callbacks. Data: Postgres or Spanner for config and tenancy; Redis Cluster for sessions, ephemeral states, and KV cache; ClickHouse or BigQuery for analytics. Streaming and Queues: Kafka for logs, events, and long■running tool orchestration. Platform: Kubernetes, service mesh with SPIFFE identities; Terraform or Pulumi and Helm; Argo CD and Argo Rollouts.

Storage Strategy

Hot Path Redis for session and short conversation windows, KV cache for model context reuse. Postgres for config and tenancy. Feature store for routing signals optional. Retrieval Vector DB sharded by tenant and region; document store in object storage with signed URLs; metadata in relational store. Periodic compaction and re■chunking. Cold/Analytics ClickHouse or BigQuery for logs, traces, and usage; Parquet in object storage for raw events with retention and time travel. Secrets and Keys KMS backed encryption; per■tenant CMKs where purchased; envelope encryption of sensitive fields. Scaling Approach Frontends Stateless and autoscaled by p95 latency and open streams. Connection affinity optional to optimize caches. Retrieval Shard by tenant and by space; cache nearest neighbor indices per hot corpus; pre■compute top■k for popular queries. Model Serving Autoscale GPU replicas on queue depth and decode utilization; admission control and burst queues; configure batch sizes and max context. KV cache shard per replica; enable continuous batching and speculative decoding to increase tokens per second. Multi■Region Default serve in■region; thin global router chooses nearest healthy cell, accounting for residency; cross■region failover for stateless only unless tenant policy allows replication.

Data Models

(abridged) Tenant tenant_id, name, plan, region_policy, cmk_ref optional, limits. APIKey key_id, tenant_id, scopes, created_at, last_used_at, status. Conversation conv_id, tenant_id, user_id pseudonymous, settings json, created_at, updated_at, retention_ttl. Message msg_id, conv_id, role, content pointer, tokens_in, tokens_out, safety_flags, created_at. RetrievalDoc doc_id, tenant_id, source_uri, embeddings_ptr, metadata json, created_at, updated_at. EmbeddingChunk chunk_id, doc_id, vector, metadata json, created_at. Tool tool_id, tenant_id, spec json, timeout_ms, version, created_at. UsageRecord tenant_id, conv_id, request_id, tokens_in, tokens_out, tool_ms, model_ms, cost_units, created_at.

Latency Budgets and SLAs

Router and auth 20 ms combined p95. Safety prefilter 20 ms p95 (rules first, ML on sample). Retrieval 100-250 ms p95 including rerank if used. Queue wait < 150 ms p95 at target load; elastic headroom to keep p99 < 400 ms. First token < 300-800 ms p95 depending on context length; stream rate 30-150 tokens per second. Availability 99.95 percent streaming success; 99.9 percent admin and write APIs.

Key Algorithms and Workflows

Routing Policy matrix per tenant maps request attributes to model families and tool sets; score based cost, latency, safety risk; fallback graph when providers degrade. Retrieval Chunk with semantic boundaries; embed using small, fast models; ANN search with HNSW or IVF; rerank top candidates; build prompt context with citations. Speculative Decoding Draft small model proposes tokens; target model verifies; improves first■token time and stream rate when supported. Safety Two

stage filters: fast rules and regex at ingress; ML classifiers at egress on sampled tokens or full chunks; PII redaction and policy actions. Tool Use JSON tool contracts; router selects tools; retries with exponential backoff; long-running tool calls handled via async queue and callback messages. Caching Prompt and response caches keyed by normalized prompt, tenant, and model version; bounded by TTL and

invalidated on doc updates. KV cache for transformer attention reuse when supported.

Security, Reliability, Compliance Security TLS 1.3, mTLS in mesh, OAuth scopes and ABAC, per-tenant encryption keys, signed webhooks, input validation, WAF and bot protection. Secret scanning in CI, SBOMs, signed images. Privacy Minimize PII; configurable retention; DSAR APIs; residency enforcement; redaction pipelines. Reliability Multi-AZ per cell; active-active stateless tiers; durable queues; idempotent writes; chaos and DR drills. Circuit breakers and load shedding on provider failures. Compliance SOC 2 Type 2 evidence capture, GDPR/CCPA DPA, audit logs, access reviews, vendor assessments. Multi-Tenancy Tenant id in every key and metric. Namespaces per tenant; quotas for RPS and tokens; per-tenant routing, safety, and tool policies. Optional hard isolation with dedicated cells and VPCs for regulated customers.

CI/CD and Release Safety Everything as code with Terraform or Pulumi and Helm. Argo Rollouts for canary by percentage and by tenant cohorts; auto-rollback on SLO burn (p95 latency, error rate, queue lag). Contract tests for APIs and tool specs; schema registry for events. Offline shadow traffic for model upgrades.

APIs and Integration Points Public REST and WebSocket POST chat completions (streaming), POST embeddings, GET conversations, POST messages, DELETE conversation, GET usage. Admin POST tenant, POST api key, PATCH policies, POST tool registration, GET audit logs, POST connector.

Webhooks Tool callbacks, safe-completion events, billing usage posted to customer systems.

Connectors Files and URLs, cloud drives, ticketing, CRM, code repos; ingestion workers with OAuth connectors.

Observability and Operational Practices Golden Signals Request rate, error rate, p95 and p99 time to first token, stream rate, queue depth, GPU utilization, retrieval latency, cache hit ratio, safety block rate, cost per 1k tokens. Tracing End-to-end spans from gateway to model to tool; prompt id and request id threading; token-level timing. SLO-Aware Autoscaling Scale frontends on p95 and open streams; serve load shed headers when budgets burn. Budget alerts per tenant. Runbooks Provider outage, cache stampede, hot tenant, KV cache eviction storm, safety false positives, cross-region failover.

Capacity Planning and Cost Estimate Example busy region: 10-50k RPS, average output 300-700 tokens, burst 2x. GPU pool sized for peak with 30 percent headroom; enable continuous batching to raise throughput. Vector DB cluster with shards per tenant; Redis 12-24 shards for sessions and caches.

Rough monthly cost per busy region 60k-250k depending on GPU type, utilization, and retention.

Testing Strategy and Failure Drills

Unit tests for routing, safety rules, tool contracts, and caching. Property tests for token accounting and idempotency. Contract tests for public API and webhooks. Load tests for long conversations and burst traffic. Chaos drills for provider degradation, GPU node loss, cache failures, and regional isolation. DR rehearsals for object storage and logs.

Phased Build Plan

MVP three months Single region cell, streaming chat, router with two model backends, retrieval MVP, basic safety rules, Redis session store, usage metering, dashboards. Phase two six to nine months Multi-region cells, vector sharding and rerank, canary router, stronger safety ML, tool registry and async tools, cost controls and per-tenant budgets. Phase three twelve months Hard isolation options, advanced routing with speculative decoding, caching at scale, enterprise compliance features, and aggressive SLOs.

Key Choices, Trade-Offs, Risks, Mitigations GPU efficiency versus latency: batching boosts throughput but increases queue wait; tune for each tier and offer priority pools. Model diversity raises resilience but increases ops complexity; mitigate with adapters and unified logging. RAG boosts quality and reduces hallucinations but adds latency and cost; cache hot contexts and use fast rerankers. Strict safety filters

reduce risk but can block benign content; allow tenant tuning and appeal workflows. Per-tenant isolation increases cost; provide tiers from shared to dedicated cells.